Example Program Using File-System Calls

1 Basic Idea

Goal

The example program copies one UNIX file from a source path to a destination path using ordinary file-system calls.

- The program is called from the command line.
- It opens the source file.
- It creates the destination file.
- It repeatedly reads blocks from the source.
- It writes each block to the destination.
- It closes both files when finished.

Core Point

The program demonstrates how `open`, `creat`, `read`, `write`, and `close` work together in a simple file-copy command.

2 Example Command

```
copyfile abc xyz
```

- `copyfile` is the program name.
- `abc` is the source file.
- `xyz` is the destination file.
- If `xyz` already exists, it is overwritten.
- If `xyz` does not exist, it is created.
- The program must receive exactly two file-name arguments.

3 Full Program

```c
/* File copy program. Error checking and reporting is minimal. */
#include <sys/types.h>
#include <fcntl.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char *argv[]);

#define BUF_SIZE 4096
#define OUTPUT_MODE 0700

int main(int argc, char *argv[])
{
    int in_fd, out_fd, rd_count, wt_count;
    char buffer[BUF_SIZE];

    if (argc != 3) exit(1);

    in_fd = open(argv[1], O_RDONLY);
    if (in_fd < 0) exit(2);

    out_fd = creat(argv[2], OUTPUT_MODE);
    if (out_fd < 0) exit(3);

    while (1) {
        rd_count = read(in_fd, buffer, BUF_SIZE);
        if (rd_count <= 0) break;

        wt_count = write(out_fd, buffer, rd_count);
        if (wt_count <= 0) exit(4);
    }

    close(in_fd);
    close(out_fd);

    if (rd_count == 0)
        exit(0);
    else
        exit(5);
}
```

4 Header Files and Constants

- The `#include` lines bring in declarations, constants, and function prototypes.
- `<fcntl.h>` provides file-control constants such as `O_RDONLY`.
- `<unistd.h>` provides POSIX system-call interfaces such as `read`, `write`, and `close`.
- `BUF_SIZE` is set to 4096, so the program copies data in 4 KB chunks.
- `OUTPUT_MODE` is set to `0700`, which controls the protection bits for the destination file.

Good Practice

Naming constants such as `BUF_SIZE` makes the program easier to read and easier to change later.

5 Command-Line Arguments

argc and argv

`argc` gives the number of command-line strings, and `argv` is an array of pointers to those strings.

For the command:

```
copyfile abc xyz
```

the argument array is:

```
argv[0] = "copyfile";
argv[1] = "abc";
argv[2] = "xyz";
```

- `argc` should be 3.
- `argv[0]` is the program name.
- `argv[1]` is the source file name.
- `argv[2]` is the destination file name.
- If `argc != 3`, the program exits with status code 1.

6 Variables

Variable	Meaning
in_fd	File descriptor for the source file.
out_fd	File descriptor for the destination file.
rd_count	Number of bytes returned by **read**.
wt_count	Number of bytes returned by **write**.
buffer	Temporary memory buffer used to hold bytes being copied.

7 Opening and Creating Files

```
in_fd = open(argv[1], O_RDONLY);
if (in_fd < 0) exit(2);

out_fd = creat(argv[2], OUTPUT_MODE);
if (out_fd < 0) exit(3);
```

- `open(argv[1], O_RDONLY)` opens the source file for reading.

- If opening fails, `open` returns a negative value.
- `creat(argv[2], OUTPUT_MODE)` creates or overwrites the destination file.
- If creation fails, `creat` returns a negative value.
- Successful calls return small integers called file descriptors.

File Descriptor

A **file descriptor** is a small integer used by a process to identify an open file in later system calls.

8 Copy Loop

```
while (1) {
    rd_count = read(in_fd, buffer, BUF_SIZE);
    if (rd_count <= 0) break;

    wt_count = write(out_fd, buffer, rd_count);
    if (wt_count <= 0) exit(4);
}
```

- The loop repeatedly tries to read up to 4096 bytes.
- `read` stores the bytes in `buffer`.
- `rd_count` is the number of bytes actually read.
- Usually `rd_count` is 4096.
- On the final successful read, `rd_count` may be less than 4096.
- When end of file is reached, `read` returns 0.
- If `read` returns a negative value, an error occurred.

9 Why write uses rd_count

Important Detail

The program writes `rd_count` bytes, not `BUF_SIZE` bytes, because the last read may return fewer than 4096 bytes.

- Suppose only 1000 bytes remain in the source file.
- `read` returns 1000, not 4096.
- The program must write only those 1000 valid bytes.
- Writing `BUF_SIZE` would copy stale or invalid buffer contents.

10 Closing and Exit Status

```
close(in_fd);
close(out_fd);

if (rd_count == 0)
    exit(0);
else
    exit(5);
```

- Both files are closed after the copy loop ends.
- Closing releases OS open-file table entries.
- Closing may also flush buffered file data.
- If the last read returned 0, the program reached end of file normally.
- If the last read was negative, the program exits with an error code.

11 Exit Codes

Exit code	Meaning
0	Successful copy.
1	Wrong number of command-line arguments.
2	Source file could not be opened.
3	Destination file could not be created.
4	Write error.
5	Read error after the copy loop ended.

12 System Calls Used

Call	Role in the program
`open`	Opens the existing source file for reading.
`creat`	Creates or overwrites the destination file.
`read`	Reads bytes from the source file into memory.
`write`	Writes bytes from memory into the destination file.
`close`	Releases the open source and destination files.
`exit`	Terminates the process with a status code.

13 Program Limitations

- Error checking is minimal.
- Error reporting is poor because it only returns numeric status codes.
- It does not print helpful error messages.
- It assumes `write` writes all requested bytes when it succeeds.

- A production-quality program should handle partial writes.
- A production-quality program should report system error messages.

Final Point

This copy program shows the standard file-copy pattern: open source, create destination, repeatedly read and write blocks, then close both files.